ADVANCED DEVOPS LAB MANUAL

University-Style Practical Manual in Docker and Containerization

Course Title	DevOps Infrastructure and Containerization Lab
Programme	B.Tech CSE – 3rd Year
Primary Tool Stack	Docker, Docker Hub, Linux utilities, Python, Nginx, Alpine, Ubuntu
Academic Use	Laboratory Manual / Faculty Handout / Student Record Support
Coverage	From basics to advanced containerization practice
Prepared For	Department of Computer Science & Engineering

Prepared for institutional academic use

Certificate / Instructor Endorsement

This is to certify that this laboratory manual is designed for the course **DevOps Infrastructure and Containerization Lab** and is intended to support structured practical delivery for B.Tech CSE third-year students. The manual covers foundational to advanced exercises aligned with modern containerization practice, Docker-based workflows, and introductory DevOps infrastructure concepts. It may be adopted as a departmental lab guide, student record companion, or instructor support document.

Document Highlights

- 22 experiments arranged from fundamentals to advanced applications.
- Step-by-step procedures with the reason behind each command or action.
- University-friendly sections including aim, objectives, prerequisites, theory, procedure, outcome, viva, assignment, and marking scheme.
- Suitable for outcome-based education, practical evaluation, and record-book documentation.

Signatures

Prepared By	Verified By	Approved By
Signature & Date	Signature & Date	Signature & Date

Table of Contents

Experiment 01. Introduction to Containers and Docker

Experiment 02. Docker Architecture: CLI, Daemon, Engine, and Registry

Experiment 03. Docker CLI Basics and Object Types

Experiment 04. Container Lifecycle Management

Experiment 05. Process Isolation and Linux Namespaces

Experiment 06. Container Runtime: Interactive and Detached Execution

Experiment 07. Docker Images, Layers, and Writable Filesystem

Experiment 08. Building Custom Docker Images with Dockerfile

Experiment 09. Docker Registry and Docker Hub

Experiment 10. Docker Volumes and Persistent Storage

Experiment 11. Docker Networks and Connectivity

Experiment 12. Multi-Container Communication Scenario

Experiment 13. Port Mapping and Service Exposure

Experiment 14. Resource Limits with cgroups

Experiment 15. Inspecting Docker Objects

Experiment 16. Logs, Exec, Attach, and Debugging

Experiment 17. Dockerfile Optimization and Layer Caching

Experiment 18. Troubleshooting Common Docker Failures

Experiment 19. Static Web Application Deployment with Nginx

Experiment 20. Python Flask Application Containerization

Experiment 21. Mini Project: Multi-Container Student Application

Experiment 22. Industry Case-Based Troubleshooting Lab

Course Outcomes

- CO1: Explain the origin, significance, and DevOps relevance of containers.
- CO2: Use Docker CLI to pull, build, run, inspect, and manage containers.

- CO3: Understand Docker architecture, daemon behavior, and runtime concepts.
- CO4: Demonstrate process isolation, namespaces, cgroups, image layering, networks, and volumes.
- CO5: Build and distribute custom images using Dockerfiles and Docker Hub.
- CO6: Troubleshoot basic to intermediate Docker deployment issues and design simple multi-container applications.

General Lab Instructions

Instruction	Rationale
Perform the experiments in sequence.	The manual is scaffolded from basic to advanced concepts.
Do not skip the “Reason” notes.	They connect command execution to conceptual learning.
Record outputs and screenshots where appropriate.	Practical documentation strengthens reproducibility and evaluation.
Use container names carefully and clean up unused resources.	Good naming and hygiene reduce confusion in multi-step labs.
Attempt viva and assignment questions after each experiment.	Reflection reinforces learning beyond command memorization.

Suggested Weekly Plan

Week	Coverage
Week 1	Experiments 1–3
Week 2	Experiments 4–5
Week 3	Experiments 6–8
Week 4	Experiments 9–11
Week 5	Experiments 12–14
Week 6	Experiments 15–17
Week 7	Experiments 18–20
Week 8	Experiments 21–22, viva, and mini-project review

Assessment Framework

Component	Suggested Weightage
Lab Performance and Observation	40 marks
Viva and Oral Explanation	20 marks
Mini Project / Advanced Case Work	20 marks
Record Book / Documentation Quality	20 marks

Experiment 01

Introduction to Containers and Docker

Aim: To understand the origin of containers, the need for containerization, and the basic working of Docker.

Learning Objectives

- Differentiate traditional deployment, virtual machines, and containers.
- Understand why modern containerization became central to DevOps.
- Pull and run the first Docker image and container.

Prerequisites: Docker Desktop or Docker Engine installed; Internet connection for pulling images; Basic terminal usage

Theory: Traditional deployment often caused dependency conflicts because multiple applications shared the same host environment. Virtual machines improved isolation but added overhead because every VM carried a full guest operating system. Containers solved this by packaging the application and its dependencies while sharing the host kernel. Docker popularized this model by making images easy to build, distribute, and run in a consistent way across development, testing, and production systems.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Check Docker installation	docker version	Confirms that both Docker client and server are available.
2	View engine details	docker info	Displays runtime, storage, network, and system details.
3	Pull a sample image	docker pull hello-world	Downloads a small test image from Docker Hub.
4	Run the sample container	docker run hello-world	Creates and starts a container to verify the setup.
5	List local images	docker images	Shows all images currently stored on the machine.
6	List containers	docker ps -a	Shows running and stopped containers for observation.

Expected Outcome: Students can explain the basic idea of containerization and successfully run their first Docker container.

Post-Lab Viva Questions

1. Why did containers become popular after virtual machines?

2. What is the difference between an image and a container?

3. Why is hello-world commonly used as the first Docker experiment?

4. How does Docker reduce the “works on my machine” problem?

Assignment / Extension: Write a one-page comparison of traditional deployment, virtual machines, and containers with one real-world example of each.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 02

Docker Architecture: CLI, Daemon, Engine, and Registry

Aim: To understand Docker architecture and the interaction between the Docker client and daemon.

Learning Objectives

- Identify the role of Docker CLI, Docker daemon, Docker Engine, and registry.
- Understand how a user command is processed internally.
- Recognize Docker object categories.

Prerequisites: Docker installed; Terminal access

Theory: Docker follows a client-server architecture. The Docker CLI accepts user commands, while the Docker daemon performs tasks such as building images, creating containers, managing volumes, and configuring networks. Docker registries store and distribute images, with Docker Hub acting as the default public registry. This architecture supports automation and repeatability, which are essential qualities in DevOps workflows.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Show client and server versions	docker version	Separates client and server components visually.
2	Inspect the complete engine state	docker info	Shows runtime, cgroup driver, storage driver, plugins, and defaults.
3	Explore top-level help	docker --help	Introduces grouped command categories.
4	Explore object-specific help	docker image --help docker container --help docker network --help docker volume --help	Reinforces the object-oriented Docker CLI design.
5	Check daemon process on Linux/WSL	ps -ef | grep dockerd	Shows the dockerd background service managing Docker operations.

Expected Outcome: Students can describe Docker command flow from CLI to daemon and explain why Docker uses a client-server model.

Post-Lab Viva Questions

1. What is the role of the Docker daemon?

2. Why is Docker described as a client-server architecture?

3. What is the function of a registry?

4. What are Docker objects?

Assignment / Extension: Draw the Docker architecture diagram and explain what happens internally when the command docker run nginx is executed.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 03

Docker CLI Basics and Object Types

Aim: To become familiar with Docker CLI syntax and the core Docker objects: image, container, network, and volume.

Learning Objectives

- Use Docker CLI commands in object-specific form.
- Identify the purpose of each Docker object.
- Observe how these objects are listed and managed.

Prerequisites: Docker running

Theory: Modern Docker uses object-focused commands such as docker image, docker container, docker network, and docker volume. This structure makes administration more logical and helps learners understand that Docker is not only about running containers but also about managing the complete lifecycle of application packaging, connectivity, and storage.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	List images	docker image ls	Shows all images available locally.
2	List containers	docker container ls -a	Shows active and stopped containers.
3	List networks	docker network ls	Displays predefined and user-created networks.
4	List volumes	docker volume ls	Displays Docker-managed persistent storage units.
5	Pull nginx image	docker pull nginx	Adds a standard image for later experiments.

Expected Outcome: Students can distinguish among images, containers, networks, and volumes and use the CLI more systematically.

Post-Lab Viva Questions

1. Which object is used for persistent storage?

2. Which object is used for communication between containers?

3. Can a container exist without an image?

4. Why are Docker commands grouped by object type?

Assignment / Extension: Prepare a one-page chart showing Docker object, purpose, example command, and one practical use case.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 04

Container Lifecycle Management

Aim: To learn how to create, start, stop, restart, inspect, and remove containers.

Learning Objectives

- Use run, start, stop, restart, logs, and rm commands.
- Understand the lifecycle of a service container.

Prerequisites: nginx image available locally

Theory: A container is a running instance of an image. In practice, DevOps engineers must frequently start, stop, restart, inspect, and remove containers as part of testing and service operations.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Run nginx in detached mode	docker run -d --name web1 nginx	Creates a background web server container.
2	Check running containers	docker ps	Verifies that the service is active.
3	Stop the container	docker stop web1	Gracefully stops the service.
4	Start the same container again	docker start web1	Starts an already existing stopped container.
5	Restart the container	docker restart web1	Useful when reloading application state.
6	View logs	docker logs web1	Shows stdout and stderr of the containerized process.
7	Remove container	docker rm -f web1	Deletes the container when it is no longer needed.

Expected Outcome: Students can confidently manage the lifecycle of a Docker container.

Post-Lab Viva Questions

1. What is the difference between docker run and docker start?

2. Why do we use detached mode?

3. What happens to the image when a container is removed?

4. Why are logs important in operations?

Assignment / Extension: Run an Apache or Nginx container on a custom name and document each lifecycle stage with screenshots.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 05

Process Isolation and Linux Namespaces

Aim: To observe how containers isolate processes and system identities using namespaces.

Learning Objectives

- Understand PID, UTS, network, and mount isolation conceptually.
- Observe container-specific process and hostname views.

Prerequisites: Ubuntu image pulled

Theory: Containers isolate resources using Linux namespaces. This gives each container a separate process tree, hostname, network stack, and filesystem view while still sharing the host kernel.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Run Ubuntu interactively	docker run -it --name iso1 ubuntu bash	Starts an interactive shell inside the container.
2	Check hostname	hostname	Shows the container-specific identity.
3	View processes	ps -ef	Shows a limited process view inside the container.
4	Inspect from host	docker top iso1	Shows container processes from the host perspective.
5	Exit shell	exit	Ends the foreground session.

Expected Outcome: Students can explain why a container behaves like an isolated process space rather than a full machine.

Post-Lab Viva Questions

1. What is namespace isolation?

2. Why does each container have its own hostname?

3. Why does the process list differ from the host?

4. Is a container the same as a virtual machine?

Assignment / Extension: Create two Alpine containers and compare their hostnames and process lists.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2

Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 06

Container Runtime: Interactive and Detached Execution

Aim: To understand foreground and background container execution modes.

Learning Objectives

- Use interactive, detached, exec, attach, and logs commands.
- Select the correct execution mode for debugging and service deployment.

Prerequisites: Ubuntu and nginx images available

Theory: Foreground mode is useful when a user must interact directly with the process, whereas detached mode is appropriate for long-running services such as web servers and databases.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Run Ubuntu interactively	docker run -it ubuntu bash	Best for learning and debugging.
2	Exit container	exit	Stops the main shell process.
3	Run nginx in detached mode	docker run -d --name web2 nginx	Runs a service in background.
4	View logs	docker logs web2	Observes output from a detached process.
5	Open shell inside running container	docker exec -it web2 bash	Provides practical runtime debugging access.

Expected Outcome: Students can choose suitable execution modes for different container scenarios.

Post-Lab Viva Questions

1. What is detached mode?

2. Why is docker exec usually preferred over attach?

3. What is the use of logs?

4. Why does an interactive shell container often stop after exit?

Assignment / Extension: Run one interactive and one detached container and compare behavior after leaving the shell.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1

Viva performance	2

Experiment 07

Docker Images, Layers, and Writable Filesystem

Aim: To understand layered images and the writable layer added to a running container.

Learning Objectives

- Use docker history to study image layers.
- Observe temporary changes in a container writable layer.

Prerequisites: Ubuntu image available

Theory: A Docker image is composed of read-only layers. When a container is created, Docker adds a thin writable layer on top. Changes made there stay with that container only unless committed or externalized.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Inspect image history	docker history ubuntu	Shows how an image is assembled in layers.
2	Run a container	docker run -it --name layer1 ubuntu bash	Creates a shell session for experimentation.
3	Create a test file	echo "DevOps Layer Demo" > /tmp/demo.txt cat /tmp/demo.txt	Stores a file in the container writable layer.
4	Restart and verify file	docker start layer1 docker exec -it layer1 bash cat /tmp/demo.txt	Shows that data remains in the same container.
5	Remove and recreate	docker rm -f layer1 docker run -it ubuntu bash cat /tmp/demo.txt	Shows that the change is absent in a new container.

Expected Outcome: Students understand the difference between image layers and container-specific writable state.

Post-Lab Viva Questions

1. Why are image layers read-only?

2. What is a writable layer?

3. Why does data disappear when a container is removed?

4. What does docker history show?

Assignment / Extension: Explain with diagram how Docker layering supports storage efficiency.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2

Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 08

Building Custom Docker Images with Dockerfile

Aim: To build a custom Docker image for a Python application.

Learning Objectives

- Write and interpret a Dockerfile.
- Build and run a custom image.

Prerequisites: Python source file and Dockerfile prepared

Theory: Dockerfiles provide a reproducible way to create images. Each instruction typically contributes a layer and becomes part of a repeatable build pipeline.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Create project folder	mkdir pythonapp && cd pythonapp	Keeps build files organized.
2	Create app.py	print("Hello from Python Docker App")	Provides a minimal application.
3	Create Dockerfile	FROM python:3.12-slim WORKDIR /app COPY app.py . CMD ["python", "app.py"]	Describes the build and runtime instructions.
4	Build image	docker build -t pythonapp:v1 .	Creates a custom image from the Dockerfile.
5	Run container	docker run --name pyapp1 pythonapp:v1	Executes the packaged Python application.
6	Inspect history	docker history pythonapp:v1	Connects Dockerfile steps to image layers.

Expected Outcome: Students can package a small Python application into a Docker image.

Post-Lab Viva Questions

1. What does FROM do?

2. Why is WORKDIR useful?

3. What is the role of COPY?

4. What does CMD define?

Assignment / Extension: Modify the Python message, rebuild the image with a new tag, and compare both runs.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3

Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 09

Docker Registry and Docker Hub

Aim: To distribute custom images using Docker Hub or another registry.

Learning Objectives

- Tag images using registry-compatible names.
- Push and pull images for sharing.

Prerequisites: Docker Hub account

Theory: Registries allow teams to share prebuilt application artifacts. Docker Hub is the default public registry and is widely used in educational and industrial workflows.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Login	docker login	Authenticates the user to the registry.
2	Tag local image	docker tag pythonapp:v1 yourusername/pythonapp:v1	Creates a registry-style repository tag.
3	Push image	docker push yourusername/pythonapp:v1	Uploads the image for sharing.
4	Remove local tag	docker rmi yourusername/pythonapp:v1	Prepares for a fresh pull test.
5	Pull again	docker pull yourusername/pythonapp:v1	Confirms distribution and retrieval.

Expected Outcome: Students can publish and retrieve a custom image through a registry workflow.

Post-Lab Viva Questions

1. What is a Docker registry?

2. Why are tags important?

3. What is Docker Hub?

4. Why are registries useful in DevOps pipelines?

Assignment / Extension: Document the steps to share a classroom image with a teammate using Docker Hub.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3

Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 10

Docker Volumes and Persistent Storage

Aim: To understand why container storage is temporary and how volumes preserve data.

Learning Objectives

- Create and attach Docker volumes.
- Verify persistence across container removal.

Prerequisites: Ubuntu image available

Theory: Container writable layers are not a reliable place for long-term data. Docker volumes are designed for persistent and shareable storage, especially for databases and user-generated content.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Create a volume	docker volume create student_data	Creates Docker-managed persistent storage.
2	Run Ubuntu with volume	docker run -it --name vol1 -v student_data:/data ubuntu bash	Mounts the volume into the container.
3	Write data	echo "persistent file" > /data/file.txt cat /data/file.txt	Stores data inside the mounted volume.
4	Remove first container	docker rm -f vol1	Deletes the container but not the volume.
5	Reuse volume in new container	docker run -it --name vol2 -v student_data:/data ubuntu bash cat /data/file.txt	Verifies persistence.

Expected Outcome: Students can preserve and reuse data independently of container lifecycle.

Post-Lab Viva Questions

1. What is persistent storage?

2. Why should databases use volumes?

3. How is a volume different from a writable layer?

4. Can one volume be mounted in multiple containers?

Assignment / Extension: Create a second file in the same volume from another container and note the result.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2

Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 11

Docker Networks and Connectivity

Aim: To create custom networks and enable communication between containers.

Learning Objectives

- Create user-defined bridge networks.
- Connect containers and test basic connectivity.

Prerequisites: Alpine image available

Theory: User-defined bridge networks improve service grouping, name resolution, and isolation compared with the default bridge in many practical deployments.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Create network	docker network create devops-net	Creates an isolated custom bridge network.
2	Run first container	docker run -dit --name c1 --network devops-net alpine sh	Places the first container on the network.
3	Run second container	docker run -dit --name c2 --network devops-net alpine sh	Places the second container on the same network.
4	Install ping utility in c1	docker exec -it c1 sh apk update apk add iputils	Adds a simple network test tool.
5	Ping second container	ping c2	Demonstrates name-based connectivity.
6	Inspect network	docker network inspect devops-net	Shows members, subnet, and endpoint details.

Expected Outcome: Students can create custom networks and explain internal container communication.

Post-Lab Viva Questions

1. Why are user-defined bridge networks useful?

2. How do containers resolve each other by name?

3. What is network isolation?

4. When is port mapping unnecessary?

Assignment / Extension: Create a diagram showing c1, c2, and the custom network with labels.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2

Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 12

Multi-Container Communication Scenario

Aim: To demonstrate service discovery and internal communication in a multi-container setup.

Learning Objectives

- Deploy two containers on one network.
- Test internal reachability using service names.

Prerequisites: Docker network concept understood

Theory: Modern applications often use multiple cooperating services such as frontend, backend, and database containers. Internal name-based communication is therefore a core DevOps skill.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Create application network	docker network create app-net	Provides a shared communication space.
2	Run server container	docker run -dit --name server1 --network app-net python:3.12-slim bash	Creates one service node.
3	Run client container	docker run -dit --name client1 --network app-net alpine sh	Creates another node for testing.
4	Install tools in client	docker exec -it client1 sh apk update apk add iputils curl	Adds test utilities.
5	Ping server by name	ping server1	Shows internal service discovery by container name.

Expected Outcome: Students can explain how services communicate inside a shared Docker network.

Post-Lab Viva Questions

1. Why is service communication important?

2. What happens if containers are on different networks?

3. Can containers communicate without publishing ports?

4. What is internal versus external access?

Assignment / Extension: Extend the scenario by adding a third container and documenting its role.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2

Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 13

Port Mapping and Service Exposure

Aim: To expose container services to the host machine using published ports.

Learning Objectives

- Use -p to publish container ports.
- Understand host port versus container port.

Prerequisites: nginx image available

Theory: Containers communicate internally through Docker networks, but host users or external clients need published ports to access a service running inside a container.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Run nginx with port mapping	docker run -d --name webport -p 8080:80 nginx	Maps host port 8080 to container port 80.
2	Open in browser	http://localhost:8080	Validates that the service is reachable from the host.
3	Verify mapping	docker ps	Shows published ports in the container list.

Expected Outcome: Students can expose a service and correctly interpret host-to-container port mapping.

Post-Lab Viva Questions

1. What does 8080:80 mean?

2. Why is port mapping required?

3. What happens if the host port is already busy?

4. Is port mapping needed for containers on the same network?

Assignment / Extension: Start two web containers on different host ports and compare access URLs.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 14

Resource Limits with cgroups

Aim: To control CPU and memory usage of containers using Docker runtime flags.

Learning Objectives

- Apply memory and CPU limits.
- Observe container resource usage with docker stats.

Prerequisites: Ubuntu image available

Theory: Without limits, a badly behaved container can consume excessive resources and affect other workloads. Docker relies on Linux control groups, or cgroups, to enforce resource restrictions.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Run memory-limited container	docker run -it --name mem1 --memory=256m ubuntu bash	Caps memory use for the container.
2	Run CPU-limited container	docker run -it --name cpu1 --cpus=0.5 ubuntu bash	Limits the container to half a CPU.
3	Monitor in another terminal	docker stats	Shows live CPU, memory, and network usage.

Expected Outcome: Students can apply and explain runtime resource limits for safer operations.

Post-Lab Viva Questions

1. What are cgroups?

2. Why are limits important in production?

3. What does --cpus=0.5 mean?

4. Why is docker stats useful?

Assignment / Extension: Research one example where missing resource limits caused service instability.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 15

Inspecting Docker Objects

Aim: To use docker inspect for detailed metadata analysis of containers, images, networks, and volumes.

Learning Objectives

- Read Docker object metadata in JSON form.
- Use inspection for troubleshooting and understanding configuration.

Prerequisites: At least one container, image, network, and volume exist

Theory: The inspect command provides detailed runtime and configuration metadata. It is an essential tool for debugging because simple list commands often do not reveal enough context.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Inspect container	docker inspect webport	Shows complete container metadata.
2	Inspect image	docker inspect nginx	Shows image details and configuration.
3	Inspect network	docker inspect devops-net	Shows network structure and connected endpoints.
4	Inspect volume	docker inspect student_data	Shows mountpoint and driver details.

Expected Outcome: Students can retrieve detailed metadata and identify when inspection is needed.

Post-Lab Viva Questions

1. Why is docker inspect important?

2. How is inspect different from ls?

3. When should a DevOps engineer use inspect?

4. What information does it provide?

Assignment / Extension: Extract three useful fields from docker inspect output and explain their meaning.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1

Viva performance	2

Experiment 16

Logs, Exec, Attach, and Debugging

Aim: To learn practical debugging approaches for running containers.

Learning Objectives

- Use docker logs and docker exec for runtime diagnosis.
- Understand when attach is less suitable.

Prerequisites: nginx image available

Theory: Debugging containers efficiently requires examining logs, entering a running container, and verifying files, ports, and processes without disrupting the primary service.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Run debugging container	docker run -d --name debugweb nginx	Starts a target container for diagnosis.
2	View logs	docker logs debugweb	Checks application output.
3	Enter the container	docker exec -it debugweb bash	Opens an interactive management shell.
4	Explore web root	cd /usr/share/nginx/html ls	Checks files inside the running container.
5	Exit shell	exit	Returns to the host terminal.

Expected Outcome: Students can inspect logs and access a running container safely for troubleshooting.

Post-Lab Viva Questions

1. Why is docker exec preferred over attach?

2. How can logs reveal application failure?

3. What should be checked inside a container during debugging?

4. Why is interactive inspection useful?

Assignment / Extension: Replace the default Nginx index file inside the container and verify the change from the browser.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 17

Dockerfile Optimization and Layer Caching

Aim: To understand Docker build cache and write more efficient Dockerfiles.

Learning Objectives

- Observe cache reuse during repeated builds.
- Learn why instruction order matters.

Prerequisites: requirements.txt and app.py files prepared

Theory: Docker caches layers to accelerate rebuilds. Frequently changing files should be copied later in the Dockerfile so earlier expensive steps, such as dependency installation, remain cached.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Use optimized Dockerfile	FROM python:3.12-slim WORKDIR /app COPY requirements.txt . RUN pip install -r requirements.txt COPY app.py . CMD ["python", "app.py"]	Places dependency installation before application code for better cache reuse.
2	Build first time	docker build -t cache-demo:v1 .	Creates the initial cache.
3	Modify app.py and rebuild	docker build -t cache-demo:v2 .	Shows reuse of early layers.
4	Modify requirements.txt and rebuild	docker build -t cache-demo:v3 .	Demonstrates cache invalidation for dependency layer.

Expected Outcome: Students can explain why Dockerfile ordering affects build performance.

Post-Lab Viva Questions

1. What is build cache?

2. Why should requirements.txt be copied before app.py?

3. What happens when an early step changes?

4. Why does this matter in CI/CD?

Assignment / Extension: Design a poor Dockerfile ordering example and explain why it rebuilds slowly.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2

Observation / output / record quality	1
Viva performance	2

Experiment 18

Troubleshooting Common Docker Failures

Aim: To identify and solve common containerization errors in lab and industry settings.

Learning Objectives

- Diagnose container exit, port conflicts, missing images, data loss, and network issues.
- Practice systematic troubleshooting.

Prerequisites: Basic Docker familiarity

Theory: Real DevOps work involves frequent diagnosis of runtime failures. A structured troubleshooting approach reduces downtime and improves confidence when working with containers.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Container exits immediately	docker run ubuntu	Stops because no long-running foreground process exists.
2	Fix with interactive shell	docker run -it ubuntu bash	Keeps the shell process active.
3	Port conflict example	docker run -d -p 8080:80 nginx docker run -d -p 8080:80 httpd	Second container fails because the host port is already in use.
4	Resolve port conflict	docker run -d -p 8081:80 httpd	Uses a different host port.
5	Network diagnosis	Ensure both containers are attached to the same user-defined network	Required for straightforward name-based communication.

Expected Outcome: Students can identify typical failure patterns and propose corrective action.

Post-Lab Viva Questions

1. Why do containers exit immediately?

2. What causes port allocation errors?

3. Why can data disappear after redeployment?

4. How do you troubleshoot communication failure between containers?

Assignment / Extension: Create a troubleshooting table with issue, symptom, cause, and solution for five Docker errors.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2

Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 19

Static Web Application Deployment with Nginx

Aim: To deploy a custom static web page using Docker and Nginx.

Learning Objectives

- Build an image with custom HTML content.
- Expose the site on a host port.

Prerequisites: Basic Dockerfile knowledge

Theory: A static website is one of the easiest ways to demonstrate application packaging and deployment. It also introduces students to the idea of replacing default content in a base image.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Create project folder	mkdir staticweb && cd staticweb	Organizes application files.
2	Create index.html	<h1>Welcome to DevOps Lab</h1> This page is served using Docker and Nginx.	Provides simple web content.
3	Create Dockerfile	FROM nginx:latest COPY index.html /usr/share/nginx/html/index.html	Replaces the default site file.
4	Build image	docker build -t staticweb:v1 .	Packages the custom web page.
5	Run container	docker run -d --name mysite -p 8080:80 staticweb:v1	Publishes the site to the host.

Expected Outcome: Students can package and deploy a simple web page as a containerized service.

Post-Lab Viva Questions

1. Why is Nginx used here?

2. What does COPY do?

3. How is this different from running plain nginx?

4. How can the page be customized?

Assignment / Extension: Add CSS styling to the page and rebuild the image under a new tag.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2

Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 20

Python Flask Application Containerization

Aim: To containerize and run a simple Flask application.

Learning Objectives

- Package a Python web application with dependencies.
- Run the application through a published port.

Prerequisites: Python, Flask source files ready

Theory: Containerizing an application means packaging code, dependencies, and execution instructions together so that the service behaves consistently across different systems.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Create app.py	from flask import Flask app = Flask(__name__) @app.route("/") def home(): return "Hello from Flask Docker App" app.run(host="0.0.0.0", port=5000)	Defines a minimal web service.
2	Create requirements.txt	flask	Lists the required package.
3	Create Dockerfile	FROM python:3.12-slim WORKDIR /app COPY requirements.txt . RUN pip install -r requirements.txt COPY app.py . CMD ["python", "app.py"]	Builds a complete application image.
4	Build image	docker build -t flaskapp:v1 .	Packages the service.
5	Run container	docker run -d --name flaskweb -p 5000:5000 flaskapp:v1	Publishes the Flask application.

Expected Outcome: Students can package a dependency-based Python application and access it through a browser.

Post-Lab Viva Questions

1. Why must the app listen on 0.0.0.0?

2. Why is requirements.txt important?

3. Why is port 5000 published?

4. What does the Dockerfile achieve?

Assignment / Extension: Replace the route output with your name and roll number, then rebuild and test again.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 21

Mini Project: Multi-Container Student Application

Aim: To integrate image build, networking, service exposure, and storage into one coherent project.

Learning Objectives

- Design a small multi-container architecture.
- Document Docker objects used and justify design choices.

Prerequisites: Completion of previous experiments

Theory: Mini projects help students move from isolated commands to systems thinking. Even a small two-service design demonstrates architecture, packaging, connectivity, and operational planning.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Create a custom network	docker network create student-app-net	Provides service communication.
2	Build frontend image	Use a simple static frontend or Flask UI	Represents the presentation layer.
3	Build backend/service image	Use a small API or utility service	Represents processing or data layer.
4	Run both containers	Attach both to student-app-net and publish only required frontend port	Combines packaging with controlled exposure.
5	Optional: add a volume	Mount a volume if the application stores data	Introduces persistence into the design.

Expected Outcome: Students can design and deploy a small Docker-based application architecture instead of a single isolated container.

Post-Lab Viva Questions

1. Why should services be separated into multiple containers?

2. Why is a custom network useful?

3. Which service should be exposed externally?

4. When should a volume be added?

Assignment / Extension: Build a two-container project of your choice and submit architecture diagram, commands, screenshots, and explanation.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2

Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Experiment 22

Industry Case-Based Troubleshooting Lab

Aim: To solve realistic DevOps scenarios using Docker concepts and commands.

Learning Objectives

- Apply Docker concepts to case-based reasoning.
- Develop structured diagnostic and corrective action skills.

Prerequisites: Understanding of previous experiments

Theory: Case-based labs simulate professional thinking. Instead of only executing commands, students must analyze symptoms, infer causes, and choose appropriate Docker tools to solve the problem.

Step-by-Step Procedure

Step	Action	Command / Input	Reason
1	Case 1	A developer says the application works locally but fails on another machine	Use image-based packaging explanation to address dependency mismatch.
2	Case 2	Data disappears after redeployment	Recommend and demonstrate volume-based persistence.
3	Case 3	Frontend cannot reach backend API	Check network, service name, running state, and port usage.
4	Case 4	Container consumes too much RAM	Apply memory limits and monitor with docker stats.
5	Case 5	Every rebuild is slow	Reorder Dockerfile for better layer caching.

Expected Outcome: Students can reason through practical container problems and propose grounded solutions.

Post-Lab Viva Questions

1. Which Docker features solve environment inconsistency?

2. How do you preserve application data across redeployment?

3. What should be checked when two services cannot communicate?

4. How can Dockerfile design improve build speed?

Assignment / Extension: Write full solutions for all five cases in structured format: problem, analysis, commands, and final recommendation.

Suggested Marking Scheme (10 Marks)

Criterion	Marks
Aim, theory, and conceptual understanding	2
Correct execution of commands / procedure	3
Explanation of reason behind steps	2
Observation / output / record quality	1
Viva performance	2

Appendix A: Recommended Record Format

- Experiment number and title
- Aim and learning objectives
- Prerequisites and theory summary
- Commands executed and observed output
- Screenshots or terminal captures where required
- Result statement in your own words
- Viva answers and assignment response

Appendix B: Faculty Note

This manual is structured to move students from command familiarity to infrastructure reasoning. Faculty members are encouraged to discuss not only what command is used, but also what Docker object is being manipulated, why the step matters operationally, and how the same concept appears in real software delivery pipelines.